

Universidad Nacional Autónoma De México

Facultad De Ciencias 2027 - 1

Profesor: Oscar Hernández Constantino
Ayudante de teoría: Geovani Martínez Martínez
Ayudante de lab: Víctor Manuel Mendiola Montes

Damian Pedro Aguilar Gil
324058075

Actividad 1.1 - Dardos a la medida

¿Que hace cada uno de estos métodos y que valor regresa?

nextInt() se encarga de leer el dato entero introducido por el usuario. **hasNextInt()** se encarga de revisar si el dato introducido por el usuario es un entero.

¿Que pasa si se usa nextInt() y el usuario escribe algo que no es un numero entero?

Se rompe el programa y marca que el tipo de dato ingresado por el usuario no es compatible con el entero que se necesita.

¿Como puede ayudarte hasNextInt() a evitar ese problema?

hasNextInt() ayuda a verificar que el tipo de dato ingresado por el ususario es un numero entero y haciendo uso de un if podemos poner un mensaje que le diga al usuario que tipo de dato acepta nuestro programa.

Para comenzar con esta actividad retomé el código de la estimación de pi que había realizado en la práctica anterior y lo adapté para que el usuario pudiera ingresar la cantidad de dardos desde la terminal para esto importé la clase Scanner y utilicé nextInt() para guardar el número ingresado en la variable dardosTotales conservando los cálculos que ya tenía para obtener la aproximación de pi.

Después agregué las validaciones porque el programa tenía que rechazar números negativos, el cero y cualquier dato que no fuera un entero, al principio se me complicó un poco hacer que volviera a pedir el número cuando el usuario ingresaba algo incorrecto, por lo que investigué cómo utilizar una variable boolean dentro de un ciclo while y combiné esto con hasNextInt() para comprobar el tipo de dato antes de guardarlo, también utilicé entrada.next() para descartar las entradas que no fueran números enteros y evitar que el ciclo se quedara repitiendo el mismo error.

Finalmente probé el programa con cantidades válidas de dardos y después con entradas incorrectas, incluyendo texto, números negativos, cero y números demasiado grandes, comprobando que el programa mostrara los mensajes correspondientes y volviera a solicitar una cantidad válida sin detenerse.

Actividad 1.2 - Decimal a binario interactivo

Para esta actividad primero retomé lo que habíamos visto en clase sobre la conversión de números decimales a binarios, además, en una de las presentaciones de las ayudantías encontré el código que utilizaba un ciclo while para realizar esta conversión, así que lo tomé como base y primero lo trabajé por separado para entender cómo funcionaba antes de agregar todo lo relacionado con Scanner.

El procedimiento consistía en dividir repetidamente el número entre 2, obtener el residuo con % y colocarlo al principio de una variable String para ir construyendo el resultado binario, después adapté el código para que el usuario pudiera ingresar cualquier número entero no negativo, utilizando nuevamente hasNextInt() y un ciclo while para validar los datos, también agregué una condición especial para que cuando se ingresara el número cero el resultado fuera directamente 0.

La última parte consistió en hacer que el programa preguntara si se quería realizar otra

conversión sin tener que ejecutarlo nuevamente, para resolverlo investigué cómo utilizar un ciclo do while y cómo comprobar la respuesta del usuario con hasNext(), de esta manera pude hacer que tanto s como S permitieran repetir el procedimiento y que cualquier otra respuesta terminara el programa.

Para comprobar su funcionamiento realicé tres pruebas, primero convertí el número 67 y obtuve 1000011, después ingresé valores incorrectos como -67, hola y 6.7 antes de convertir correctamente el número 3, por último realicé varias conversiones consecutivas utilizando 55, 88 y 0, con lo que pude verificar que el programa repetía el proceso y manejaba correctamente los distintos casos.

Actividad 2.1 - Programa final para la Cabaña del Misterio

En esta última actividad retomé el cotizador de préstamos que había desarrollado en las prácticas anteriores, pero ahora tenía que convertirlo en un programa interactivo que permitiera registrar los datos del cliente, seleccionar distintas operaciones y terminar automáticamente bajo ciertas condiciones.

Primero agregué Scanner para solicitar el nombre completo, la clasificación del cliente, el capital, la tasa anual y el plazo del préstamo, después incorporé ciclos while y condiciones if para validar cada dato, una de las partes que tuve que investigar fue cómo identificar las clasificaciones P, E y R sin tener problemas al comparar textos, por lo que utilicé hasNext() para reconocer la clasificación antes de consumir la entrada y aplicar el ajuste correspondiente a la tasa.

Posteriormente construí el menú utilizando if y else if para separar las operaciones, conservé las fórmulas del interés simple y compuesto, y para la opción 3 reutilicé ambos cálculos para compararlos y mostrar cuál era mayor y por cuánto, mientras que para la opción 4 agregué otro ciclo de validación que permitiera modificar el capital sin volver a registrar todos los datos.

Una de las dificultades fue conseguir que el menú siguiera funcionando cuando el usuario ingresaba letras o números que no correspondían a ninguna opción, así que agregué condiciones para rechazar esas entradas y contadores para registrar las operaciones efectivas y los errores, finalmente modifiqué la condición del while principal para que el programa terminara al completar cinco operaciones o cometer tres errores, además de permitir salir con la opción 0 y mostrar un resumen con los datos solicitados.

Para las pruebas utilicé primero los datos de Robbie Valentino con un capital de $12,899, una tasa del 15 % y un plazo de 21 meses, obteniendo $3,385.99 de interés simple y $3,844.71 de interés compuesto, con una diferencia de $458.72, resultados que coincidieron con los ejemplos de la práctica.

En la segunda ejecución simulé un préstamo para comprar una laptop utilizando un capital inicial de $20,000, una tasa anual del 12 % y un plazo de 24 meses, posteriormente cambié el capital a $25,000 y continué utilizando el menú hasta completar cinco operaciones, comprobando que las nuevas cotizaciones utilizaran el capital actualizado y que el programa terminara por el límite establecido.

Por último utilicé un cliente ficticio llamado .El buen soos", con un capital de $5,000, una tasa del 10 % y un plazo de 12 meses, ingresé datos incorrectos durante el registro para

comprobar las validaciones y después introduje las opciones 9, xd6767 y -2 en el menú, con lo que el programa registró tres errores, mostró el resumen final y terminó correctamente.

Actividad 2.1 - Programa final para la Cabaña del Misterio

Conclusión de la practica.

Esta práctica me gustó bastante porque siento que aprendí a utilizar mucho mejor ‘Scanner‘, especialmente los métodos ‘next()‘ y ‘hasNext()‘ para recibir y validar datos, lo que más se me complicó fue implementar la opción 4 del cotizador, no porque fuera difícil cambiar el capital, sino porque tenía que asegurarme de que las demás operaciones utilizaran el nuevo valor y siguieran funcionando correctamente.

Mi avance.

Al principio del proyecto prácticamente no sabía nada y me confundía con cosas que ahora me parecen bastante sencillas, pero conforme avanzaron las prácticas fui entendiendo mejor cómo funciona Java y cómo relacionar lo aprendido, además, siento que la temática de Gravity Falls hizo todo más ligero y entretenido, en general me siento bastante bien y satisfecho con lo que he logrado hacer desde la práctica 2 hasta ahora.

No fue facil.

Encontrar y corregir errores se volvió un poco más complicado porque todo el programa está dentro de ‘main‘ y conforme agregaba cosas era más difícil ubicar dónde estaba el problema, también noté que repetí varias partes del código, principalmente al mostrar los datos del cliente, calcular los intereses y validar entradas, aunque al final pude organizarlo lo suficiente para que funcionara correctamente.